

SECOND PART – ROS SERVICES, PARAMS AND LAUNCHER

We have the following nodes:

- **info_user**: Manages user input and publishes it.
- **game_node**: Manages the game phases and game logic.
- **control_node**: Listens for keyboard input and sends control commands.
- **result_game**: Collects the results of the game and displays or logs them.

Exercise Description:

- Node Game_node:

1.1 Service 1 GetUserScore: This service must return the percentage of the score when it receives the user's name.

- Create a service with the name *"user_score"* with the type **"GetUserScore"**
- GetUserScore.srv

```
"GetUserScore.srv"  
string username  
---  
int64 score
```

1.2 Service 2 SetGameDifficulty: This service must change the difficulty of the game. You can only change the difficulty of the game if you are on the start screen (phase1).

- Create a service with the name *"difficulty"* with the type **"SetGameDifficulty"**
- SetGameDifficulty.srv

```
"SetGameDifficulty.srv"  
string change_difficulty  
---  
bool success
```

- The service request can be *"easy"*, *"medium"* or *"hard"*. And the response can be *"True"*, if the game is in the phase1 or *"False"* if the game isn't in the phase1
- If the game stay in the phase1 you need to change the difficulty to the level that you have in the request.

2. **NODE:Result_node:** This node has a client that sends to the *user_score service* the name of the user. Then, It print in the terminal the percentage score it received.

Create the parameters:

In the game_node create the following parameters

- **user_name:**
 - Type: string
 - Function: Store the user's name.
- **change_player_color:**
 - Type: int64
 - Function: Change the player color.

Available colours:

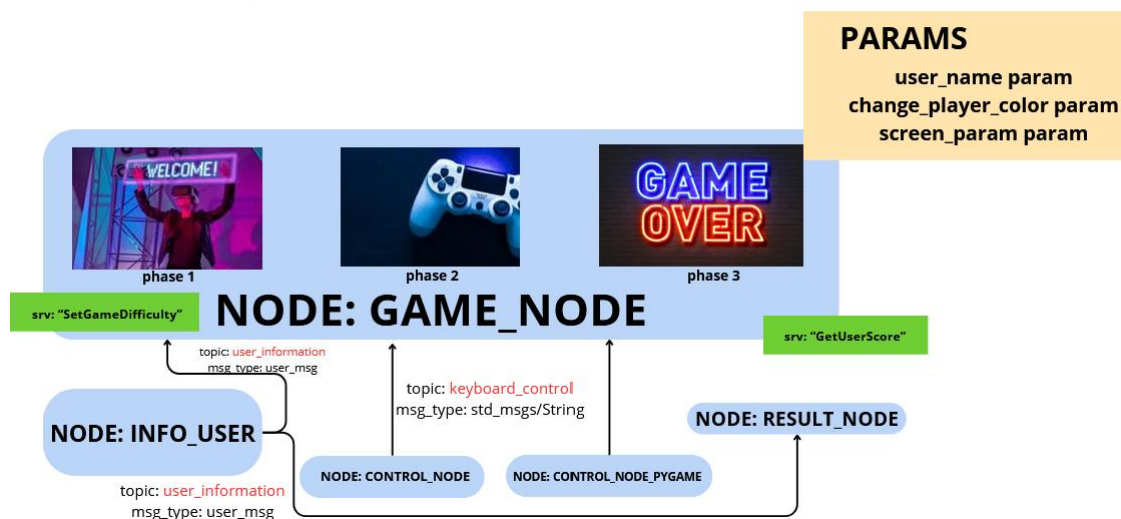
1: Red.

2: Purple.

3: Blue.

- **screen_param**
 - Type: string
 - Function: Show the game phase (phase1,phase2,phase3)

Create a launcher with the nodes to launch the complete game.
Working in Pairs - Second PART OF THE ROS FINAL WORK



Technical Requirements

1. Documentation:

- Update the README.md file that explains how to run each node.

- Include instructions for installing dependencies, running each node, and an overview of the node-to-node communication process.

2. Save the changes

- Upload to the github repository in the **ROS branch**

Requirements for the final work

- 1- The first part of the final work (part1 ROS1) – Publisher, Subscriber and custom msg
- 2- The second part of the final work – services, params, and launcher
- 3- Date: 11/12/2025
- 4- Upload all the code to github
- 5- Upload Final Work Report in AG.
 - A zip with a video showing the launching of the nodes and the general functionality of the work.